

Exercicios_PyTorch_Lista_2_RESOLVIDO

July 22, 2026

1 LISTA DE EXERCÍCIOS 2: PyTorch Tensores

1.1 Parte 2 - Avançado

Dificuldade: Intermediária a Avançada **Tempo estimado:** 60-90 minutos **Exercícios:** 10 (Níveis 5-7)

1.1.1 Pré-requisito

Complete **LISTA DE EXERCÍCIOS 1** antes de começar

1.1.2 Objetivos

- Trabalhar com múltiplas dimensões
- Broadcasting e transposição
- Indexação e slicing avançado
- Resolver problemas complexos integrando conceitos

1.2 Setup

```
[1]: import torch
import numpy as np

print(f'PyTorch versão: {torch.__version__}')
print(' Pronto para começar!')
```

PyTorch versão: 2.12.0+cu126

Pronto para começar!

2 NÍVEL 5: Operações Multi-Dimensionais

Broadcasting, transposição e operações em múltiplos eixos

2.1 EXERCÍCIO 5.1: Broadcasting

Tarefa: 1. Crie: - A = torch.tensor([[1], [2], [3]]) (3x1) - B = torch.tensor([[1, 2, 3, 4]]) (1x4) 2. Some: C = A + B 3. Qual é o shape de C? 4. Imprima C 5. Explique por que funcionou

```
[4]: # SEU CÓDIGO AQUI:
A = torch.tensor([[1], [2], [3]])
B = torch.tensor([[1, 2, 3, 4]])

C = A + B
print(C.shape)
print(C)

'''

O soma de um vetor coluna com um vetor linha resulta em uma matriz,
onde cada elemento da matriz é a soma do elemento correspondente do vetor
↪coluna com o elemento correspondente do vetor linha.

[[1+1, 1+2, 1+3, 1+4],    [[2, 3, 4, 5],
 [2+1, 2+2, 2+3, 2+4],    = [3, 4, 5, 6],
 [3+1, 3+2, 3+3, 3+4]]    [4, 5, 6, 7]]

'''
```

```
torch.Size([3, 4])
tensor([[2, 3, 4, 5],
        [3, 4, 5, 6],
        [4, 5, 6, 7]])
```

```
[4]: '\n\n0 soma de um vetor coluna com um vetor linha resulta em uma matriz, \nonde
cada elemento da matriz é a soma do elemento correspondente do vetor coluna com
o elemento correspondente do vetor linha.\n\n[[1+1, 1+2, 1+3, 1+4],    [[2, 3,
4, 5],\n [2+1, 2+2, 2+3, 2+4],    = [3, 4, 5, 6],\n [3+1, 3+2, 3+3, 3+4]]    [4,
5, 6, 7]]\n\n'
```

2.2 EXERCÍCIO 5.2: Transposição

Tarefa: 1. Crie: `t = torch.tensor([[1, 2, 3], [4, 5, 6]])` 2. Transpor usando: `- t.T - t.transpose(0, 1)` 3. Qual é o novo shape? 4. São iguais?

```
[12]: t = torch.tensor([[1, 2, 3], [4, 5, 6]])

t_T = t.T
t_tr = t.transpose(0, 1)

print(t_T)
print(t_tr)
print(t_T.shape)
print(torch.equal(t_T, t_tr))
```

```
tensor([[1, 4],
        [2, 5],
```

```

        [3, 6]])
tensor([[1, 4],
        [2, 5],
        [3, 6]])
torch.Size([3, 2])
True

```

2.3 EXERCÍCIO 5.3: Soma ao longo de eixo

Tarefa: 1. Crie: `t = torch.tensor([[1, 2, 3], [4, 5, 6]])` (2x3) 2. Some ao longo dos eixos: - `t.sum(dim=0) → shape?` - `t.sum(dim=1) → shape?` 3. Imprima cada resultado 4. Qual é o significado?

```

[15]: # SEU CÓDIGO AQUI:
t = torch.tensor([[1, 2, 3], [4, 5, 6]])
print(t.shape)

d0 = t.sum(dim=0)
d1 = t.sum(dim=1)

print(f"d0: {d0}, d1:{d1}")
print("Na dimensão 0, a soma é feita ao longo das colunas, resultando em um
↳vetor com o mesmo número de colunas que a matriz original. Na dimensão 1, a
↳soma é feita ao longo das linhas, resultando em um vetor com o mesmo número
↳de linhas que a matriz original.")

```

```

torch.Size([2, 3])
d0: tensor([5, 7, 9]), d1:tensor([ 6, 15])
Na dimensão 0, a soma é feita ao longo das colunas, resultando em um vetor com o
mesmo número de colunas que a matriz original. Na dimensão 1, a soma é feita ao
longo das linhas, resultando em um vetor com o mesmo número de linhas que a
matriz original.

```

2.4 EXERCÍCIO 5.4: Concatenação

Tarefa: 1. Crie dois tensores 2x2 2. Concatene ao longo de diferentes eixos: - `torch.cat([t1, t2], dim=0) → shape?` - `torch.cat([t1, t2], dim=1) → shape?` 3. Imprima e compare

```

[18]: # SEU CÓDIGO AQUI:
t1 = torch.tensor([[1, 2], [3, 4]])
t2 = torch.tensor([[5, 6], [7, 8]])

print(torch.cat((t1, t2), dim=0).shape) # (4, 2)
print(torch.cat((t1, t2), dim=1).shape) # (2, 4)

```

```

torch.Size([4, 2])
torch.Size([2, 4])

```

3 NÍVEL 6: Indexação e Slicing Avançado

3.1 EXERCÍCIO 6.1: Indexação básica

Tarefa: 1. Crie: `t = torch.arange(12).reshape(3, 4)` 2. Acesse: - `t[0]` → primeira linha - `t[0, 0]` → primeiro elemento - `t[-1, -1]` → último elemento 3. Modifique: `t[0, 0] = 999`

```
[23]: # SEU CÓDIGO AQUI:
t = torch.arange(12).reshape(3, 4)
print(t)

print(f"Primeira linha: {t[0]}")
print(f"Primeiro elemento: {t[0, 0]}")
print(f"Último elemento: {t[-1, -1]}")

t[0, 0] = 999
print(f"Primeiro elemento modificado: {t[0, 0]}")
print(t)
```

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
Primeira linha: tensor([0, 1, 2, 3])
Primeiro elemento: 0
Último elemento: 11
Primeiro elemento modificado: 999
tensor([[999,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

3.2 EXERCÍCIO 6.2: Slicing

Tarefa: 1. Crie: `t = torch.arange(20).reshape(4, 5)` 2. Realize slices: - `t[0:2]` → primeiras 2 linhas - `t[:, 0:3]` → todas linhas, primeiras 3 colunas - `t[:, 2, ::2]` → linhas/colunas pares 3. Imprima cada resultado

```
[28]: # SEU CÓDIGO AQUI:
t = torch.arange(20).reshape(4, 5)
print(t)
print(f"Primeiras 2 linhas: {t[0:2]}")
print(f"Todas as linhas, primeiras 3 colunas: {t[:, :3]}")
print(f"Linhas e colunas pares: {t[:, 2, ::2]}")
```

```
tensor([[ 0,  1,  2,  3,  4],
        [ 5,  6,  7,  8,  9],
        [10, 11, 12, 13, 14],
        [15, 16, 17, 18, 19]])
Primeiras 2 linhas: tensor([[0, 1, 2, 3, 4],
                            [5, 6, 7, 8, 9]])
```

```
Todas as linhas, primeiras 3 colunas: tensor([[ 0,  1,  2],
        [ 5,  6,  7],
        [10, 11, 12],
        [15, 16, 17]])
Linhas e colunas pares: tensor([[ 0,  2,  4],
        [10, 12, 14]])
```

3.3 EXERCÍCIO 6.3: Indexação booleana

Tarefa: 1. Crie: `t = torch.tensor([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])` 2. Crie máscara: `mask = t > 5` 3. Acesse: `t[mask]` 4. Filtre múltiplas: `(t > 3) & (t < 8)`

```
[31]: # SEU CÓDIGO AQUI:
t = torch.tensor([1,2,3,4,5,6,7,8,9,10])
print(t)

mask = t > 5
print(mask)

print(t[mask])
filtrar = t[(t > 3) & (t < 8)]
print(filtrar)
```

```
tensor([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
tensor([False, False, False, False, False,  True,  True,  True,  True,  True])
tensor([ 6,  7,  8,  9, 10])
tensor([4, 5, 6, 7])
```

4 NÍVEL 7: Desafios Integrados

Combine múltiplos conceitos para resolver problemas complexos

4.1 DESAFIO 7.1: Manipular matriz

Tarefa: 1. Crie matriz 4x5 com números aleatórios (0-10) 2. Calcule: - Soma de cada linha - Média de cada coluna - Elemento máximo - Elemento mínimo 3. Normalize a matriz (valores 0-1)

```
[35]: # SEU CÓDIGO AQUI:
t = torch.randint(0, 11, (4, 5), dtype=torch.float32)
print(t)

print(f"Soma por linha: {t.sum(dim=1)}")
print(f"Média por coluna: {t.mean(dim=0)}")
print(f"Máximo: {t.max().item()}")
print(f"Mínimo: {t.min().item()}")

t_norm = (t - t.min()) / (t.max() - t.min())
```

```
print(f"Normalizada (0-1):\n{t_norm}")
```

```
tensor([[10.,  1.,  7.,  0.,  9.],
        [ 8.,  2.,  6.,  8.,  1.],
        [ 6.,  4.,  4.,  4.,  6.],
        [ 5.,  7.,  9.,  1.,  1.]])
Soma por linha: tensor([27., 25., 24., 23.])
Média por coluna: tensor([7.2500, 3.5000, 6.5000, 3.2500, 4.2500])
Máximo: 10.0
Mínimo: 0.0
Normalizada (0-1):
tensor([[1.0000, 0.1000, 0.7000, 0.0000, 0.9000],
        [0.8000, 0.2000, 0.6000, 0.8000, 0.1000],
        [0.6000, 0.4000, 0.4000, 0.4000, 0.6000],
        [0.5000, 0.7000, 0.9000, 0.1000, 0.1000]])
```

4.2 DESAFIO 7.2: Python PyTorch

Tarefa: 1. Crie lista: `[[1, 2, 3], [4, 5, 6], [7, 8, 9]]` 2. Converta para tensor 3. Adicione 10 4. Multiplique por 2 5. Calcule transposta 6. Converta de volta para lista Python

```
[42]: # SEU CÓDIGO AQUI:
lista = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
t = torch.tensor(lista)

t_add = t + 10
t_mul = t_add * 2
t_T = t_mul.T
resultado = t_T.tolist()

print(resultado)
```

```
[[22, 28, 34], [24, 30, 36], [26, 32, 38]]
```

4.3 DESAFIO 7.3: Clone vs Referência

Tarefa: 1. Crie tensor: `t = torch.randn(3, 3)` 2. Crie 3 versões: - `t1 = t` (referência) - `t2 = t.clone()` (cópia) - `t3 = t.detach()` (detach) 3. Modifique: `t1[0, 0] = 999` 4. Verifique o que mudou em cada uma 5. Explique as diferenças

```
[53]: # SEU CÓDIGO AQUI:
t = torch.randn(3, 3)

t1 = t
t2 = t.clone()
t3 = t.detach()

t1[0, 0] = 999
```

```

print(f"t  [0,0]: {t[0,0].item()}")    # 999 - t1 É t
print(f"t1 [0,0]: {t1[0,0].item()}")  # 999
print(f"t2 [0,0]: {t2[0,0].item()}")  # valor original - cópia independente
print(f"t3 [0,0]: {t3[0,0].item()}")  # 999 - compartilha storage

print('''
t1 = t - atribuição Python pura. t1 e t são o mesmo objeto (t1 is t → True).
↳ Qualquer mutação in-place reflete em ambos.
t2 = t.clone() - aloca memória nova e copia os dados. Independente de t. Se t
↳ tiver requires_grad=True, o clone preserva o grad_fn (continua no grafo
↳ computacional).
t3 = t.detach() - compartilha o mesmo storage (mesma memória de dados), mas
↳ cria um novo tensor desvinculado do grafo de autograd. Mutações in-place em
↳ t ou t3 afetam ambos, mas t3.requires_grad é False independentemente de t.
''')
```

```

t  [0,0]: 999.0
t1 [0,0]: 999.0
t2 [0,0]: 0.17900998890399933
t3 [0,0]: 999.0
```

t1 = t - atribuição Python pura. t1 e t são o mesmo objeto (t1 is t → True). Qualquer mutação in-place reflete em ambos.

t2 = t.clone() - aloca memória nova e copia os dados. Independente de t. Se t tiver requires_grad=True, o clone preserva o grad_fn (continua no grafo computacional).

t3 = t.detach() - compartilha o mesmo storage (mesma memória de dados), mas cria um novo tensor desvinculado do grafo de autograd. Mutações in-place em t ou t3 afetam ambos, mas t3.requires_grad é False independentemente de t.